Frontend · concept board

# Next.js

App Router, server components, rendering strategies, caching

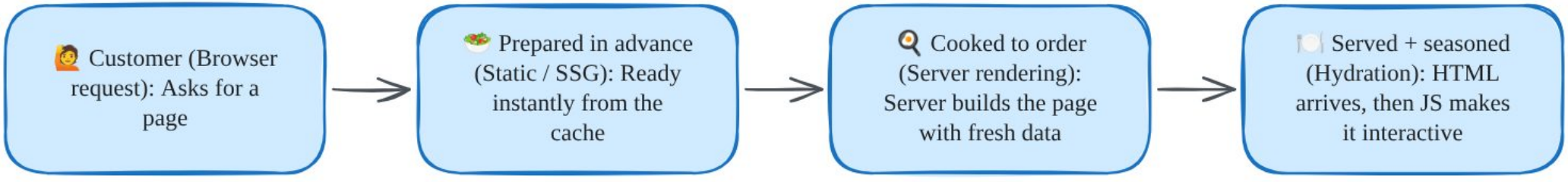
1. What Next.js adds to React
2. Server and Client Components
3. Rendering strategies
4. Data fetching, caching and mutations
5. Built-in features

Draw on any slide, then Present (arrow keys) or export the board as a PDF.

# 1. What Next.js adds to React

### Definition

Next.js is a React framework: file-based routing, server rendering, data fetching, caching, API routes, image/font optimisation and bundling in one tool.

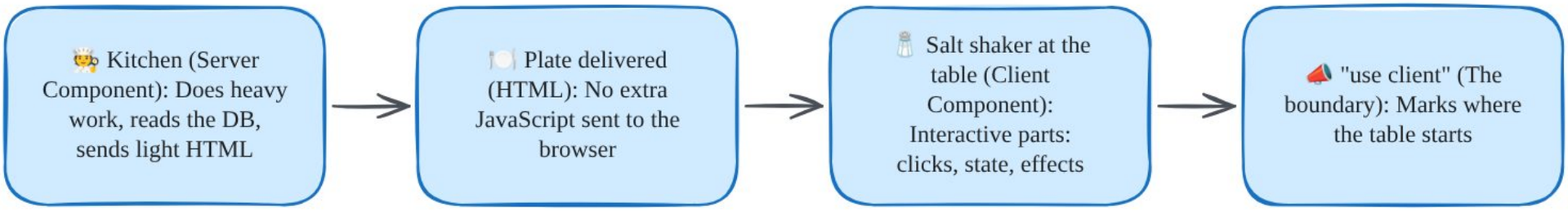



Think of Next.js like a restaurant with prep cooks

### App Router file conventions

```
app/
  layout.tsx          shared UI (wraps every page), keeps state across navigation
  page.tsx            the "/" route
  loading.tsx         Suspense fallback while the route loads
  error.tsx           error boundary for the segment ('use client')
  not-found.tsx       404 UI
  blog/
    page.tsx          "/blog"
    [slug]/page.tsx   "/blog/:slug" — dynamic segment
  (marketing)/about/  route group: organises files, not part of the URL
  api/users/route.ts  Route Handler: GET/POST/… → JSON
```

# 2. Server and Client Components




Think of server and client components like kitchen and table

| Server Component (default) | Client Component ('use client') |
|---|---|
| Runs on the server only; its code is not sent to the browser | Rendered on the server for HTML, then hydrated in the browser |
| Can be async, read the DB or secrets directly | Can use state, effects, event handlers, browser APIs |
| No useState, onClick, window | Can't be async; can't import server-only code |

### Fetch on the server, interact on the client

```
// app/products/page.tsx — Server Component
import { AddToCart } from './add-to-cart'

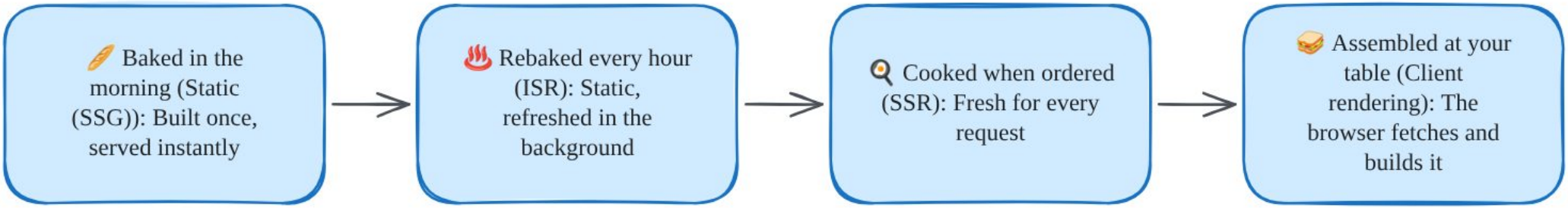
export default async function Page() {
  const products = await db.product.findMany()   // runs on the server
  return products.map((p) => (
    <div key={p.id}>{p.name} <AddToCart id={p.id} /></div>
  ))
}

// app/products/add-to-cart.tsx
'use client'
export function AddToCart({ id }: { id: string }) {
  const [added, setAdded] = useState(false)
  return <button onClick={() => setAdded(true)}>{added ? 'Added' : 'Add'}</button>
}
```

### Tip

Push 'use client' down to the leaves (buttons, forms) so most of the page stays server-rendered with zero JS.

# 3. Rendering strategies

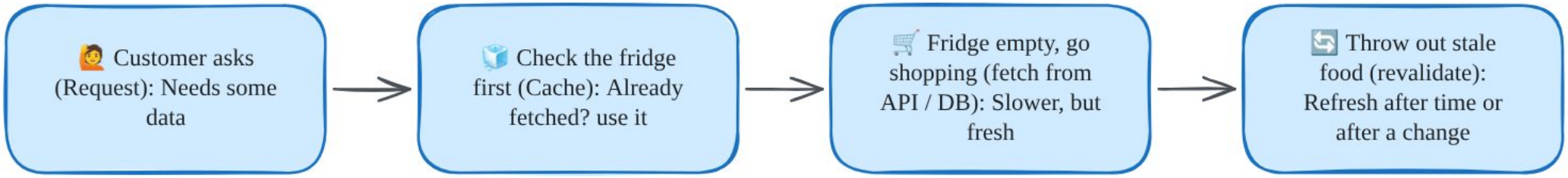



Think of rendering modes like a bakery

| Strategy | HTML made | Good for |
|---|---|---|
| CSR | in the browser | dashboards behind login |
| SSR (dynamic) | on every request | personalised or always-fresh pages |
| SSG (static) | at build time | marketing pages, docs, blogs |
| ISR | at build, then regenerated after a time or on demand | product pages, large catalogs |
| Streaming | shell first, slow parts stream in via Suspense | pages with slow data |

- Hydration = React attaching event handlers to server-rendered HTML.
- Hydration mismatch: server and client render different output (dates, window, random values).
- Partial prerendering: a static shell is served instantly, and dynamic holes stream in.

# 4. Data fetching, caching and mutations




Think of caching like a fridge in a cafe

Server Action for a form

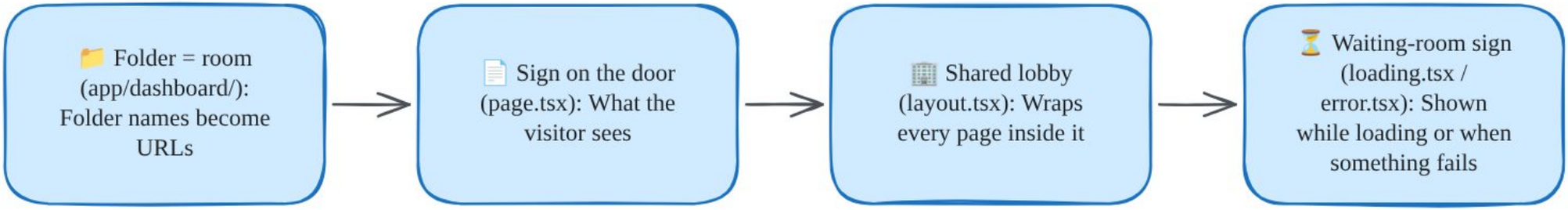
```
// app/todos/actions.ts
'use server'
import { revalidatePath } from 'next/cache'

export async function addTodo(formData: FormData) {
  const title = String(formData.get('title') ?? '').trim()
  if (!title) return { error: 'Title is required' }   // validate on the server
  await db.todo.create({ data: { title } })
  revalidatePath('/todos')                            // refresh cached data
}

// app/todos/page.tsx
<form action={addTodo}><input name="title" /><button>Add</button></form>
```

- Server Actions are public endpoints — always validate input and check auth inside them.
- Cache invalidation: revalidatePath, revalidateTag (on-demand) or time-based revalidation.
- Newer versions make caching explicit ('use cache', Cache Components); request APIs like cookies(), headers() and params are async.
- Route Handlers (route.ts) for webhooks and public APIs.

# 5. Built-in features




Think of the App Router like a building directory

| Feature | Why |
|---|---|
| next/link | client-side navigation + prefetching |
| next/image | resizing, lazy loading, modern formats, no layout shift |
| next/font | self-hosted fonts, no layout shift |
| Metadata API | export const metadata / generateMetadata for SEO tags |
| Middleware / proxy | run before a request: auth redirects, rewrites, i18n |
| generateStaticParams | pre-build dynamic routes |

### Common mistake

Reading window or localStorage during render in a Client Component — it also renders on the server. Read browser-only values in effects or event handlers.

### Revise

App Router files = routes. Server Components by default, 'use client' only for interactivity.
SSG/ISR/SSR/streaming per page. Server Actions + revalidate for mutations; validate everything on the server.

# Rapid-fire Q&A (1/2)

### Server Components vs Client Components?

Server Components run only on the server (can be async, access the DB, ship no JS). Client Components ('use client') can use state, effects and event handlers and are hydrated in the browser.

### SSR vs SSG vs ISR?

SSR renders on every request; SSG renders at build time; ISR serves static pages and regenerates them after a time or on demand.

### What is hydration?

React attaching event handlers and state to server-rendered HTML so it becomes interactive.

### What causes a hydration mismatch?

Server and client rendering different output — dates, random values, window checks or browser-only data during render.

### What are Server Actions?

Async server functions ('use server') you can call from forms or client code to mutate data; they must validate input and check auth.

### What does loading.tsx do?

It becomes the Suspense fallback for the route segment while its content loads.

# Rapid-fire Q&A (2/2)

### Why use next/image?

Automatic resizing, lazy loading, modern formats and reserved dimensions that prevent layout shift.

### How do you invalidate cached data after a mutation?

revalidatePath or revalidateTag (on demand), or time-based revalidation.

### Where should 'use client' go?

As low in the tree as possible — on the interactive leaves — so most of the page stays server-rendered.